ELEC 475

Lab 1 – MLP Autoencoder

Connor Leung 20322809

Ryan Su 20344856

Model

The model we used was a MLP autoencoder with that has a variable bottleneck size, applied to images from the FashionMNIST dataset. The autoencoder model uses the Rectified Linear Unit (ReLU) activation function with input layers of 784 nodes and an output layer of 784 nodes. The hidden layers were built with N/2 nodes, in this case, 392 nodes in hidden layers 1 and 2. A sigmoid function was used at the end of the decoder (between hidden layer 2 and the output) so that the output range matches the target range. Each 28x28 image is flatted into the 784 input nodes with pixel values in the range of [0, 1]. The final model is created as follows: 784→392→z→392→784.



Figure 1: Diagram of the Different Layers of the Autoencoder Neural Net

Throughout the lab, we tested different bottlenecks with 8, 32, and 128 nodes that is in between the two layers. The parameter count per bottlenecks are shown below.

Bottleneck Size	Parameter Count
8	622,504
32	641,344
128	716,704

Table 1: Parameters per Bottleneck Size

Training

The MLP autoencoder was trained using the Adaptive Moment Estimation (Adam) optimization algorithm, with PyTorch's MSELoss (mean squared error function) as the loss function. The training also used a ReduceLROnPlateau scheduler, which reduces the learning rate when the training stops improving for 10 epochs. Lastly, Xavier uniform initialization was used for the weights and the biases were set to 0.01. The reported training loss for each epoch is the mean of the per-batch MSE over all batches. Weights were saved after every epoch, so each .pth file holds the epoch-30 weights.

Outputs

Below are the loss vs. epoch training curves the multilayer perceptrons (MLP) autoencoder models. Each model was trained for 30 epochs. A common trend during training was that the more nodes in the bottleneck, the lower the loss over the epochs.

With a bottleneck of 128, the loss is below 0.01, whereas with a the bottleneck of 8, the loss is higher than 0.015.

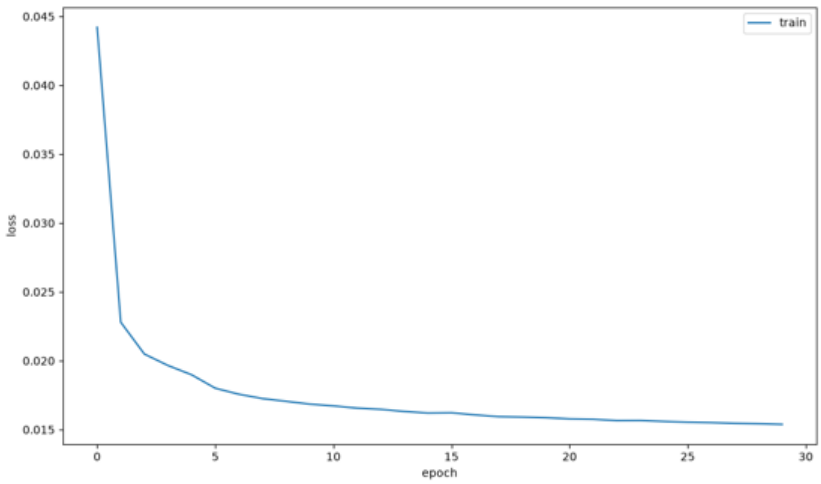


Figure 2: Loss vs. Epoch with Bottleneck of 8

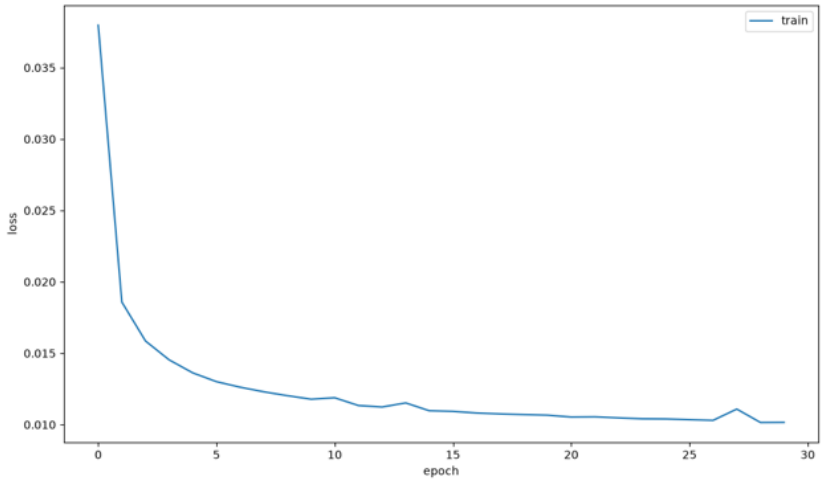


Figure 3: Loss vs. Epoch with Bottleneck of 32

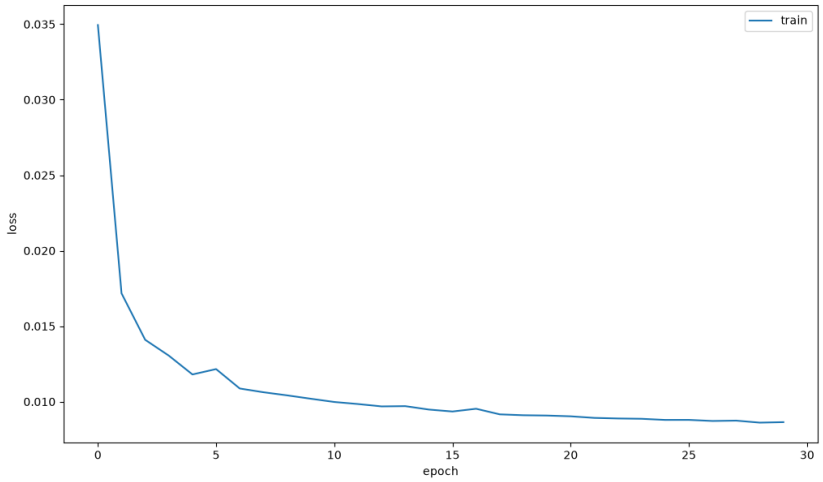


Figure 4: Loss vs. Epoch with Bottleneck of 128

There is a ~34% decrease in loss when we had the bottleneck is increased 4x (8 to 32), but only a with marginal decrease of ~15% when it is increased 4x again (32 to 128). This shows a decreasing trend in loss as the bottleneck size increases.

Bottleneck Size	Loss
8	0.01538
32	0.01016
128	0.008641

Table 2: Mean Squared Error per Bottleneck Size

Below is a figure showing the same images from the FashionMNIST dataset after passing through each autoencoder. There is more detail with the larger bottlenecks, which supports the trend seen in the table and the loss vs. epoch graphs.

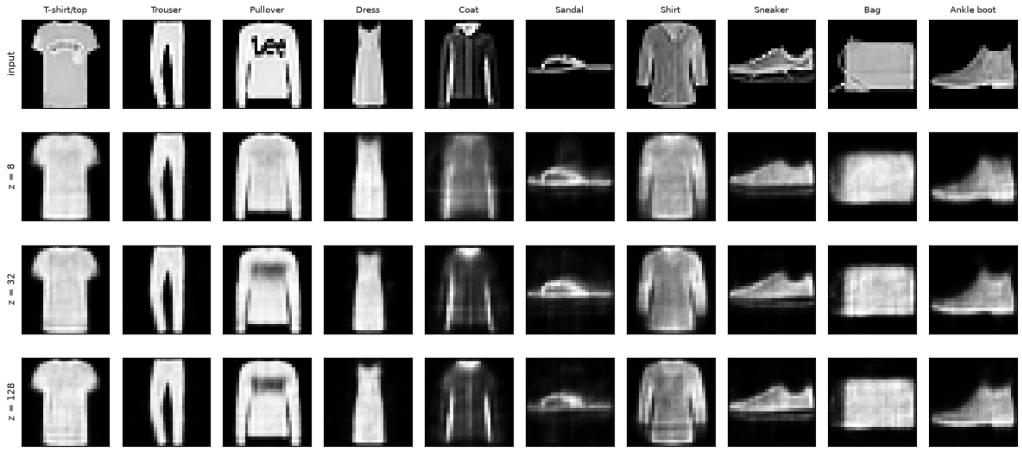


Figure 5: Comparison of Images Across Different Bottlenecks

The image below is the interpolation between a pair of trousers and a boot. Using the defined equation $z = (1-a)z_1 + a\,z_2$, we combine the two images in latent space by taking a weighted average the two bottleneck vectors until one image transforms into the other (a going from 0 to 1 over 8 steps). This gives us the effect of the trousers are fading into the boot.

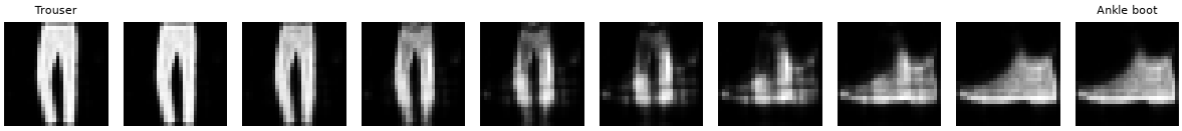


Figure 6: Interpolation with Bottleneck of 32

Here are the reconstructions from each bottleneck.

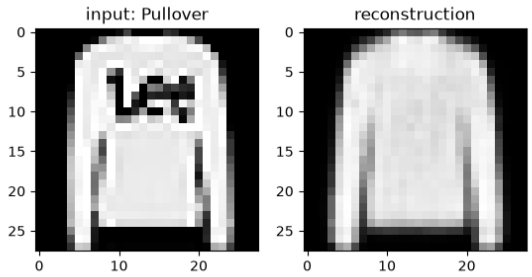


Figure 7: Output from 8 Node Bottleneck



Figure 8: Output from 32 Node Bottleneck



Figure 9: Output from 128 Node Bottleneck

Below, noise was introduced into the image using the PyTorch's built-in uniform random function (torch.rand), scaled by 0.3, and the result was clamped down to [0, 1], the valid pixel range, using the MLP.32 model. The result was a brighter image, and the reconstruction reflects this in the text area, where the normal reconstruction had more dark detail. Since the model was not primarily trained on noisy images, it is to be expected that it is farther from the original image.

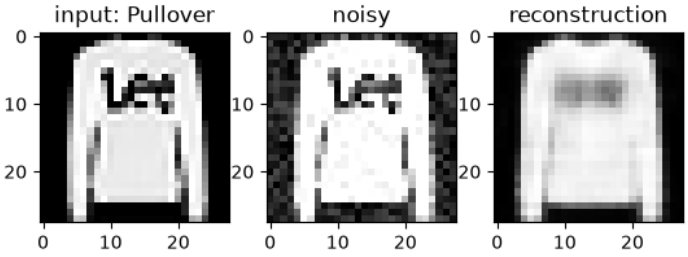


Figure 10: Denoise Output with Bottleneck of 32

Analysis

a)

The information that survives compression is the large details, such as the shape of the clothing item and any large details on it (text on a shirt gets compressed down to a blob of the same colour; see Figures 7–9). When reconstructing with a bottleneck of 32, the image keeps some detail of the text on the shirt, which is a significant amount of retained information compared to the bottleneck of 8, where the image does not have a dark spot where the text should be. This supports the improvement from the 8 to the 32 node bottleneck described in Table 2, where the 32 node bottleneck has 34% less loss than the 8 node bottleneck. There is a less noticeable difference from 32 to 128 nodes, as the level of detail stays about the same, with 128 nodes retaining slightly more of the colour detail, which reflects the 15% lower loss from 32 to 128 nodes.

The classes that reconstruct the worst at the 32 node bottleneck are the sandal, bag and shirt (see Figure 5). The sandal is made of thin straps with gaps in between, and these get filled in so the sandal becomes a solid blob of colour. The bag and shirt have a lot of different shapes and patterns within their class, meaning the model learns a general shape for them and loses the smaller details that make each one different. This is the same reason the text on the shirt is lost, since the small details are not kept at the bottleneck.

b)

It works well for noise removal because it only retains the larger features of an image. The hidden layers and bottleneck keep the main information of the image at each step, meaning that, for example, the sleeve of a shirt will be preserved since it is a larger feature, while text on a shirt is lost since it is a smaller feature. Since the model is trained on the FashionMNIST dataset, it can only reconstruct images that look like the training images. Noise is random and has no pattern, so it is not something that can be stored in the 32 bottleneck nodes and is left out when the image is compressed. The decoder tries to rebuild the image to the closest clothing item it has learned, which is why the noise is removed This is shown in Figure 10, where the reconstruction from the noisy input looks almost the same as the normal reconstruction in Figure 8, with the text area being slightly brighter.

c)

Bottleneck size affects reconstruction quality based on how much information is retained at the bottleneck nodes. Since the network moves from a full image to fewer nodes until the bottleneck, the information at each layer is compressed to the most dominant features. The trend in Table 2, where going from 8 to 32 nodes gives a 34% decrease in loss while going from 32 to 128 gives only a 15% decrease, supports that the reconstruction is only as good as the information given at the bottleneck.

d)

During interpolation, the transition from trouser to boot looks like the boot is emerging from the background of the trouser. It looks as if the boot is hidden behind the trouser, and in the middle the boot starts to appear while the trouser fades away. While it looks like the trouser is fading into the boot, the pixels are not actually blending since interpolation happens in latent space. If it was pixel blend, the intermediary images would be the trouser image on top of the boot. In Figure 6, the images are a noisy image that does not look like either clothing item. This is because each step between z_1 and z_2 is a new vector that the decoder is not trained on which is why it results in a noisy and malformed image. This suggest that the bottleneck space is not a seamless transition between classes, meaning the points between trouser and boot do not decode to an actual piece of clothing.